

Video Recording Script — Full Walkthrough

Memory-Based Detection of Golden Ticket & SIDHistory Abuse in Active Directory

P Rahul · R23MTC09 · REVA University · MTech Capstone

How to use this: The **[ON SCREEN]** lines tell you what to open / click / point to.

The **[SAY]** lines are your voiceover — read them aloud, verbatim, in a calm steady pace.

Target total length **10–12 minutes**. Record scenes separately and stitch them — you do NOT have to do it in one take.

SECTION 0 — Before you hit record (setup, 20–30 min ahead)

1. **Power on the DC VM and make sure it will NOT suspend** (this keeps AMA/log ingestion alive).
2. Run `C:\SecurityScripts\START-LAB.cmd`. Wait for "lab is UP".
3. Confirm ingestion is live: Sentinel → Logs → `SecurityEvent | summarize max(TimeGenerated)` (< 5 min old).

If AMA is down: restart Arc stack `himds → GCArcService → ExtensionService`, then **wait ~14 min**.

4. **Pre-run the attack ONCE** so incidents already exist (you'll re-run it on camera as a demo, but the finished evidence must already be there — detection takes 5–10 min and you can't wait on video).

5. **Open these browser tabs, in this order (left to right):**

- Tab 1: `architecture-diagram.html` (open the local file)
- Tab 2: Azure Portal → Microsoft Sentinel → **HybridDetectionWS** → **Overview**
- Tab 3: Sentinel → **Incidents** (filter: Last 24 hours)
- Tab 4: Sentinel → **Logs** (with the "Capstone Evidence" saved queries visible)
- Tab 5: Sentinel → **Workbooks** → *Attack Corroboration Dashboard (MTech)*
- Tab 6: Sentinel → **Workbooks** → *Capstone Evidence (MTech)*
- Tab 7: <https://capstonedash824f.z13.web.core.windows.net/> (the live dashboard)
- Tab 8: Sentinel → **MITRE ATT&CK**

6. Open a **PowerShell / terminal window**, maximised, with the attack command ready to paste (Scene 4).

7. **Recording tips:** 1920×1080, hide desktop icons, close notifications/Teams/email, zoom the browser to 110–125% so text is readable on video, and do a 20-second mic test first.

8. Confirm the correct Azure tenant is shown (top-right): **agpandian@hotmail.com**.

SCENE 1 — Title & introduction (~45 sec)

[ON SCREEN] A title slide OR your webcam. Show: project title, your name, SRN R23MTC09, REVA University.

[SAY]

"Hello. I'm Rahul, and this is my MTech capstone project: *Memory-Based Detection of Golden Ticket and

SIDHistory Abuse in Active Directory*.

Golden Ticket and SIDHistory are two of the stealthiest attacks against Active Directory — they let an attacker impersonate a domain administrator and persist almost invisibly. The problem is that traditional,

log-only security tools can only *infer* these attacks from behaviour, and they never see the forged ticket itself, because it lives inside the domain controller's memory.

In this project I built a hybrid detection framework on Microsoft Sentinel that does something different: it detects both attacks, and it recovers the actual forged ticket from memory — then corroborates that memory evidence against the logs. Let me walk you through it."

SCENE 2 — Architecture overview (~1 min 15 sec)

[ON SCREEN] Tab 1 — the architecture diagram. Slowly move your cursor along the three detection paths as you name them.

[SAY]

"Here is the architecture. My detection framework has three independent paths, all feeding into Microsoft

Sentinel.

The first is the **log path**. My domain controller is connected to Azure through Azure Arc, and the Azure Monitor Agent forwards Kerberos security events — ticket requests, privileged logons — into a Log Analytics workspace.

The second is the **SIDHistory path**. A script on the domain controller inventories Active Directory for accounts carrying privileged SID-History entries and sends those records directly to Log Analytics.

The third — and this is my main contribution — is the **memory forensics path**. I capture the domain controller's LSASS process memory, upload it to Azure storage, and a dedicated analysis virtual machine examines that memory offline to recover the forged Kerberos ticket itself.

All three paths converge in Sentinel, where analytics rules turn them into incidents, and a corroboration

dashboard cross-confirms the log evidence against the memory evidence. Now let me show it working, live."

SCENE 3 — Show the platform is live (~40 sec)

[ON SCREEN] Tab 2 — Sentinel Overview. Point to the events/incidents counters. Then Tab 4 — Logs — run:
`SecurityEvent | summarize count(), max(TimeGenerated) .`

[SAY]

"This is my live Microsoft Sentinel workspace. You can see security events are actively flowing in from the domain controller — the most recent event is just moments old, which confirms the log pipeline is healthy. Everything you'll see from here is real data from a live environment, not screenshots."

SCENE 4 — Launch the attack (live) (~1 min 15 sec)

[ON SCREEN] The terminal. Paste and run the Golden Ticket attack command. Let the mimikatz output scroll.

[SAY]

"Now I'll launch the attack. I'm playing the role of an attacker who has already compromised the domain.

First, using a DCSync operation, I extract the secret key of the KRBTGT account — the master key that signs every Kerberos ticket in the domain.

Next, with that key, I forge a Golden Ticket for the Administrator account, giving it Domain Admin and Enterprise Admin privileges, and I inject it directly into memory — this is a Pass-the-Ticket attack.

Notice the forged ticket is being accepted; the attacker is now operating as a full domain administrator.

Finally, I trigger a capture of the domain controller's memory, so my framework can analyse it.

The attack is done. Now let's see the framework catch it — in two independent ways."

[TIP] After this scene, STOP recording briefly. In real life detection takes several minutes — resume recording once your pre-staged incidents are visible, so the video flows without dead air.

SCENE 5 — Detection fired: the incidents (~1 min)

[ON SCREEN] Tab 3 — Incidents (Last 24h). Point to the three incident types. Click into the "Volatility Memory Analysis" incident to show its details.

[SAY]

"Within minutes, Sentinel has raised incidents automatically. Look at the three types here.

The first is the **Golden Ticket** incident, from the log path — it detected a service ticket request

that had no preceding ticket-granting-ticket, which is the classic footprint of an injected Golden Ticket.

The second is the **SIDHistory** incident, flagging an account carrying a privileged SID it should not have.

And the third — the important one — is the **memory analysis** incident. This one didn't come from logs at

all; it came from analysing the domain controller's memory. Let me show you what it found."

SCENE 6 — The recovered artifact (the differentiator) (~1 min 30 sec)

[ON SCREEN] Tab 4 — Logs → Queries → "Capstone Evidence" → double-click ***01 - Memory: Recovered Forged Ticket*** → Run. Point to the columns: the .kirbi filename, TargetAccount = Administrator, LifetimeDays = 3650.

[SAY]

"This is the heart of my project. A log-only tool can tell you an attack *probably* happened. My framework shows you the actual weapon.

Here, recovered directly from the domain controller's memory, is the forged Kerberos ticket itself. You can

see the ticket file, and that it was forged for the **Administrator** account.

And look at this column — the ticket's lifetime is **three thousand six hundred and fifty days**. That's

about ten years. A genuine Kerberos ticket lasts only around ten hours. A ten-year lifetime is the

unmistakable signature of a forged, mimikatz-generated Golden Ticket.

This is forensic proof — the artifact — not an inference. No log-only tool can produce this."

SCENE 7 — Corroboration: the two tracks agree (~1 min 15 sec)

[ON SCREEN] Tab 5 — Attack Corroboration Dashboard. Point to the top **CONFIRMED — Log + Memory** panel,
then the two side-by-side tracks below it.

[SAY]

"Now here is where it all comes together — my corroboration dashboard.

On the left is the **log track**: the behavioural evidence, the ticket requests seen in the Kerberos logs.

On the right is the **memory track**: the recovered forged ticket, with its ten-year lifetime.

These are two completely independent sources of evidence — one from Windows event logging, one from

physical memory forensics. My framework correlates them on the account identity.

Because both tracks independently implicate the *same* account — Administrator — the verdict at the top

reads **CONFIRMED — Log plus Memory**.

This is the core contribution: I don't just raise an alert and hope it's real. I confirm the attack two

independent ways — the behaviour *and* the artifact — which makes a false positive almost impossible."

SCENE 8 — The overview dashboards (~1 min)

[ON SCREEN] Tab 6 — Capstone Evidence workbook (scroll through KPIs, Kerberos charts, SIDHistory table, memory CRITICAL table, MITRE table). Then Tab 7 — the **live cloud dashboard** (full-screen it, F11).

[SAY]

"This is my Capstone Evidence workbook, which brings the whole picture together — detection counts across

all three paths, the Kerberos event timeline, the SIDHistory findings, and the memory-forensics detections,

all mapped to the MITRE ATT&CK framework.

And finally, this is a live cloud dashboard I built and deployed to Azure. It updates automatically and

stays online even when the lab is powered off — a single-pane view of the entire detection story: the

confirmed verdict, the recovered ticket, and the coverage. This is the kind of at-a-glance board a security

operations analyst would actually use."

SCENE 9 — MITRE mapping & results (~50 sec)

[ON SCREEN] Tab 8 — Sentinel MITRE ATT&CK page (show the highlighted techniques).

[SAY]

"Every detection in my framework is mapped to the MITRE ATT&CK knowledge base — Golden Ticket under

T1558.001, Pass-the-Ticket, SID-History Injection, and credential access from LSASS.

For validation, I built a labelled benchmark for the SIDHistory detector and measured a precision, recall, and F1 score of one-point-zero across forty samples — with no false positives. And detection time dropped

from a roughly forty-five-minute manual baseline to just a few minutes, automated."

SCENE 10 — Conclusion (~50 sec)

[ON SCREEN] Back to the title slide or your webcam, or the corroboration "CONFIRMED" panel.

[SAY]

"To summarise: existing log-only tools stop at an inference — they tell you an attack *probably* happened.

My hybrid framework goes further. It recovers the actual forged ticket from domain controller memory, and

corroborates that artifact against the logs, turning 'probably an attack' into 'confirmed, two independent

ways.'

The memory-forensics detection path is the novel element — it produces tamper-resistant forensic evidence

that behavioural, log-only detection simply cannot.

Thank you for watching. I'm happy to take any questions."

SECTION 11 — Honesty guardrails (so you never overclaim on camera)

Say these ONLY as written; do not exaggerate — an examiner reviewing the video will notice.

- **Do NOT say** "no tool can detect this" — say "no *log-only* tool can produce the artifact." (Good EDR/MDI detect it behaviourally.)
- **Do NOT name a specific memory tool as 'Volatility' recovering the ticket** — say "memory forensics / memory analysis recovers the ticket." (Standard Volatility 3 can't; the wording on screen is neutral on purpose.)
- **Do NOT quote 87%, 90%, or 100% accuracy** — only the measured benchmark: P/R/F1 = 1.00 on 40 samples.
- **Do NOT claim capture is auto-triggered by the incident** — it's scheduled/triggered; incident-driven auto-capture is stated as future work.
- If a path is down while recording, **re-record that scene later** rather than narrating around a broken screen.

SECTION 12 — After recording

- Run `C:\SecurityScripts\STOP-LAB.cmd` to save Azure credits (the live dashboard stays up).
- Edit: trim the wait after Scene 4; add on-screen text labels for each detection path if you like.
- Keep a copy of the final video alongside `Demo-Steps.pdf` and `Viva-QA-Notes.pdf`.